

大数据算法第三次作业

2025 年 6 月 14 日

Problem 1. 把最优传输问题转化为标准线性规划问题，并写出其对偶问题。

证明. 首先回顾线性规划, 原始问题 (Primal Problem) 的标准形式为:

$$\begin{aligned} \min_x \quad & c^\top x \\ \text{满足} \quad & Ax = b \\ & x \geq 0 \end{aligned}$$

其对偶问题变为:

$$\begin{aligned} \max_y \quad & b^\top y \\ \text{满足} \quad & A^\top y \leq c \end{aligned}$$

设源分布为 $\alpha \in \mathbb{R}^m$, 目标分布为 $\beta \in \mathbb{R}^n$, 代价矩阵为 $C \in \mathbb{R}^{m \times n}$ 。最优传输问题可表述为:

$$\begin{aligned} \min_{\pi} \quad & \langle C, \pi \rangle = \sum_{i=1}^m \sum_{j=1}^n C_{ij} \pi_{ij} \\ \text{s.t.} \quad & \pi 1_n = \alpha \quad (\text{行约束}) \\ & \pi^\top 1_m = \beta \quad (\text{列约束}) \\ & \pi_{ij} \geq 0, \quad \forall i, j \end{aligned} \tag{1}$$

将其转化为标准线性规划形式:

$$\begin{aligned} \min_{\pi} \quad & \text{vec}(C)^\top \text{vec}(\pi) \\ \text{s.t.} \quad & A \text{vec}(\pi) = b \\ & \text{vec}(\pi) \geq 0 \end{aligned} \tag{2}$$

其中:

- $\text{vec}(\pi) \in \mathbb{R}^{mn}$ 是将 π 按列向量化的结果
- $A = \begin{bmatrix} I_m \otimes 1_n^\top \\ 1_m^\top \otimes I_n \end{bmatrix} \in \mathbb{R}^{(m+n) \times mn}$

$$\bullet b = \begin{bmatrix} \alpha \\ \beta \end{bmatrix} \in \mathbb{R}^{m+n}$$

原始线性规划的对偶问题为：

$$\begin{aligned} \max_{f, g} \quad & f^\top \alpha + g^\top \beta \\ \text{s.t.} \quad & f_i + g_j \leq C_{ij}, \quad \forall i, j \end{aligned} \quad (3)$$

其中 $f \in \mathbb{R}^m$ 和 $g \in \mathbb{R}^n$ 是对偶变量，分别对应于原始问题中的行约束和列约束。 $\square$

Problem 2. 令 $X = \mathbb{R}^d$ ，并定义 $\mathcal{H}$ 为 X 上的所有轴对齐矩形（axis-parallel boxes）构成的集合。具体来说：

$$\mathcal{H} = \{h_{a,b} \mid a, b \in X\}$$

其中分类器 $h_{a,b}(x)$ 定义为：

$$h_{a,b}(x) = \begin{cases} 1 & \text{如果 } a_i \leq x_i \leq b_i \text{ 对所有 } i = 1, \dots, d \text{ 成立,} \\ -1 & \text{其他情况.} \end{cases}$$

比如，在二维平面上，假如 $a = (-1, -1)$ ， $b = (1, 1)$ ，那么矩形 $h_{a,b}$ 就表示 $((-1, -1), (-1, 1), (1, 1), (1, -1))$ 四个点构成的矩形。

证明 $\mathcal{H}$ 的 VC dimension 是 $2d$

证明. 为了证明 $\mathcal{H}$ 的 VC dimension 是 $2d$ ，我们需要证明存在一个包含 $2d$ 个点的集合能被 $\mathcal{H}$ 打散，但是不存在一个包含 $2d+1$ 个点的集合能被 $\mathcal{H}$ 打散。

我们首先证明存在一个包含 $2d$ 个点的集合能被 $\mathcal{H}$ 打散，假设集合 V 包含 $2d$ 个点，每个点对应一个坐标的 1 或者 -1，即 $V = \{e_1, \dots, e_d, -e_1, \dots, -e_d\} \subseteq X$ ，其中对于 $\forall i, j \in [d]$,

$$(e_i)_j = \begin{cases} 1 & \text{如果 } j = i, \\ 0 & \text{否则.} \end{cases}$$

下面说明，对于 V 的任何一个子集 S ，都能找到 $h_{a,b} \in \mathcal{H}$ ，使得 $V \cap \{x \mid h_{a,b}(x) = 1\} = S$ 。令

$$S_1 = \{-e_i \mid -e_i \in S, i \in [d]\}, \quad S_2 = \{e_i \mid e_i \in S, i \in [d]\}$$

定义

$$a = \sum_{-e_i \in S_1} (-e_i), \quad b = \sum_{e_i \in S_2} e_i$$

对于 $\forall e_i \in V, i \in [d]$ ，考虑以下四种情况：

- (i) 如果 $-e_i \in S$ 且 $e_i \in S$ ，则 $a_i = -1, b_i = 1$ ，因此 $h_{a,b}(-e_i) = 1, h_{a,b}(e_i) = 1$ ；
- (ii) 如果 $-e_i \in S$ 且 $e_i \notin S$ ，则 $a_i = -1, b_i = 0$ ，因此 $h_{a,b}(-e_i) = 1, h_{a,b}(e_i) = -1$ ；
- (iii) 如果 $-e_i \notin S$ 且 $e_i \in S$ ，则 $a_i = 0, b_i = 1$ ，因此 $h_{a,b}(-e_i) = -1, h_{a,b}(e_i) = 1$ ；

(iv) 如果 $-e_i \notin S$ 且 $e_i \notin S$, 则 $a_i = 0, b_i = 0$, 因此 $h_{a,b}(-e_i) = -1, h_{a,b}(e_i) = -1$ 。

因此, V 是可以被 $\mathcal{H}$ 打散的。故 $\mathcal{H}$ 的 VC 维至少为 $|V| = 2d$ 。然后我们证明不存在一个包含 $2d+1$ 个点的集合能被 $\mathcal{H}$ 打散

考虑任意大小为 $2d+1$ 的点集 $U \subseteq \mathbb{R}^d$ 。通过下面的过程找到这些点在每一维上的最小值和最大值, 然后使用它们作为边界构建轴对齐矩形 (axis-parallel box):

1. 初始化 $a \leftarrow 0, b \leftarrow 0, S \leftarrow \emptyset$

2. 对于每个维度 $i = 1, \dots, d$:

- 在 U 中找到两个点 $u_1, u_2 \in \mathbb{R}^d$, 使得

$$(u_1)_i = \min_{u \in U} u_i, \quad (u_2)_i = \max_{u \in U} u_i$$

(即在第 i 维上找到最小值和最大值)

- 设置 $a_i \leftarrow (u_1)_i, b_i \leftarrow (u_2)_i$
- 更新 $S \leftarrow S \cup \{u_1, u_2\}$

3. 返回 a, b, S

根据上述过程, 可知 $S \subseteq U$ 且 $|S| \leq 2d$ 。下面证明不存在 a, b 使得 $U \cap \{x \mid h_{a,b}(x) = 1\} = S$ 。

用上述过程得到的 a, b 构建轴对齐矩形时:

$$\{x \mid h_{a,b}(x) = 1\} = U$$

S 中的点恰好在矩形的边界上, 因此 $U \setminus S$ 中至少有一个点位于矩形内部。考虑所有可能的 a, b :

- (i) 若轴对齐矩形比上述构建的小, 则无法覆盖 S 中的所有点 (因为原构建使 S 恰在边界);
- (ii) 若轴对齐矩形比上述构建的大, 则 $\{x \mid h_{a,b}(x) = 1\} = U$, 即覆盖全部 U 。

因此, 不存在大小为 $2d+1$ 的点集能被 $\mathcal{H}$ 打散。

综上, 轴对齐矩形假设空间 $\mathcal{H}$ 的 VC 维为 $2d$ 。 □

Problem 3. 证明平面上的凸多边形的 VC dimension 是无穷

证明. 为了证明平面上的凸多边形的 VC dimension 是无穷的, 我们需要证明对于任意的正整数 n , 存在一个包含 n 个点的集合, 对这个集合点的任意一种标记方式, 都存在一个凸多边形能够正确分类这些点。因此我们可以构造这样一个 n 元点集: n 个点均匀的分布在一个圆上。对于这些点集的一种标记方式, 我们按照顺时针方式 (或逆时针) 顺次连接正例点, 这构成一个凸多边形, 并且所有正例点在这个凸多边形内, 负例点都在这个凸多边形外。因此, 这个 n 元点集可以被凸多边形打散。由 n 的任意性知, 凸多边形的 VC dimension 是无穷的 □

Problem 4. 设 $a_1, a_2, \dots, a_n$ 为一个符号流, 其中每个符号是集合 $\{1, \dots, m\}$ 中的一个整数。

1. 均匀随机选择：设计一个算法，从流中均匀随机选择一个符号。你的算法需要多少内存？
2. 加权随机选择：设计一个算法，以与 a_i^2 成正比的概率选择一个符号。

解. (1) 采用蓄水池采样算法 (2) 采用加权蓄水池采样算法 $\square$

Problem 5. 设 k -median 的优化目标为 $f(P, C)$ ，相应的距离度量为 g 。 $A = \{a_1, \dots, a_m\}$ 是一个 α, β -近似解，即 $f(P, A) \leq \alpha f(P, C_{opt})$ ，且 $m \leq \beta k$ 。以每个 α_j 为圆心，使用以 $2^t R$ 为半径的若干同心圆来划分空间，其中 $R = \frac{1}{\alpha} f(P, A)$ ， $t = 0, 1, \dots, \phi$ ， $\phi = \log \alpha n$ 。对于每个 j, t ，设同心圆环内部的点集为 N_j^t ，我们在每个 N_j^t 中都取 $x = \Theta(\frac{1}{\epsilon_0^2} \log \frac{1}{\lambda})$ 个点组成 coreset。那么对一个固定的 k -median 解 C ，依一定的概率，我们有 $|\sum_{p \in S_j^t} \frac{|N_j^t|}{x} g(p, C) - \sum_{p \in N_j^t} g(p, C)| \leq \epsilon_0 2^{t+1} R |N_j^t|$ (Hoeffding Bound).

试证明：

$$|\sum_{j=1}^m \sum_{t=0}^{\phi} \sum_{p \in S_j^t} \frac{|N_j^t|}{x} g(p, C) - \sum_{j=1}^m \sum_{t=0}^{\phi} \sum_{p \in N_j^t} g(p, C)| \leq \Theta(\epsilon_0) n f(P, A)$$

(提示：课上证明了若只考虑 $t \geq 1$ 层的同心圆环内的点集 N_j^t ，不等式右侧 $\leq 4\epsilon_0 n f(P, A) = \Theta(\epsilon_0) n f(P, A)$ ，这个结论成立。现在需证明在考虑 $t = 0$ 层的点集 N_j^0 的情况下，该结论依然成立)

证明.

在不等式右侧，把 $t = 0$ 层与 $t \geq 1$ 层的同心圆环分开考虑，第 0 层用到性质： $R = \frac{1}{\alpha} f(P, A)$

$$\sum_{j=1}^m \sum_{t=0}^{\phi} \epsilon_0 2^{t+1} R |N_j^t| = 4\epsilon_0 \sum_{j=1}^m (\frac{R}{2} |N_j^0| + \sum_{t=1}^{\phi} 2^{t-1} R |N_j^t|) \quad (4)$$

$$= 4\epsilon_0 \sum_{j=1}^m (\frac{f(P, A)}{2\alpha} |N_j^0| + \sum_{t=1}^{\phi} 2^{t-1} R |N_j^t|) \quad (5)$$

$$\leq 4\epsilon_0 (\frac{n}{2\alpha} f(P, A) + n f(P, A)) \quad (6)$$

$$= \Theta(\epsilon_0) n f(P, A) \quad (7)$$

$\square$

Problem 6. 证明（关于欧氏 k -means 问题的）coresets 满足下面的可组合性质 (composability)：

令 $A_1, A_2 \subseteq \mathbb{R}^d$ 是两个互不相交的集合。假设集合 S_1 及权重函数 $w : S_1 \rightarrow \mathbb{R}$ 和集合 S_2 及权重函数 $w : S_2 \rightarrow \mathbb{R}$ 分别是 A_1 和 A_2 的 (k, ϵ) -coresets。那么 $S_1 \cup S_2$ 及函数 $w_1 + w_2 : S_1 \cup S_2 \rightarrow \mathbb{R}$ 是 $A_1 \cup A_2$ 的 (k, ϵ) -coreset。

注：这里 $w_1 + w_2$ 的定义如下：

$$(w_1 + w_2)(x) = \begin{cases} w_1(x), & \text{如果 } x \in S_1 \setminus S_2, \\ w_2(x), & \text{如果 } x \in S_2 \setminus S_1, \\ w_1(x) + w_2(x), & \text{如果 } x \in S_2 \cap S_1. \end{cases}$$

证明. 因为 A_1, A_2 互不相交, 因此对任意 $C \subseteq \mathbb{R}^d, |C| \leq k$, 有:

$$\begin{aligned}
D(A_1 \cup A_2, C) &= \sum_{x \in A_1} \min_{c \in C} \|x - c\|^2 + \sum_{x \in A_2} \min_{c \in C} \|x - c\|^2 \\
&= D(A_1, C) + D(A_2, C), \\
D(S_1 \cup S_2, w_1 + w_2, C) &= \sum_{x \in S_1 \cup S_2} (w_1 + w_2)(x) \cdot \min_{c \in C} \|x - c\|^2 \\
&= \sum_{x \in S_1 \setminus S_2} w_1(x) \cdot \min_{c \in C} \|x - c\|^2 + \sum_{x \in S_2 \setminus S_1} w_2(x) \cdot \min_{c \in C} \|x - c\|^2 \\
&\quad + \sum_{x \in S_1 \cap S_2} (w_1(x) + w_2(x)) \cdot \min_{c \in C} \|x - c\|^2 \\
&= \sum_{x \in S_1} w_1(x) \cdot \min_{c \in C} \|x - c\|^2 + \sum_{x \in S_2} w_2(x) \cdot \min_{c \in C} \|x - c\|^2 \\
&= D(S_1, w_1, C) + D(S_2, w_2, C).
\end{aligned}$$

因此,

$$\begin{aligned}
&|D(A_1 \cup A_2, C) - D(S_1 \cup S_2, w_1 + w_2, C)| \\
&= |(D(A_1, C) + D(A_2, C)) - (D(S_1, w_1, C) + D(S_2, w_2, C))| \\
&= |(D(A_1, C) - D(S_1, w_1, C)) + (D(A_2, C) - D(S_2, w_2, C))| \\
&\leq |D(A_1, C) - D(S_1, w_1, C)| + |D(A_2, C) - D(S_2, w_2, C)| \\
&\leq \varepsilon D(A_1, C) + \varepsilon D(A_2, C) \\
&= \varepsilon D(A_1 \cup A_2, C).
\end{aligned}$$

根据定义, $S_1 \cup S_2$ 及函数 $w_1 + w_2 : S_1 \cup S_2 \rightarrow \mathbb{R}$ 是 $A_1 \cup A_2$ 的 (k, ε) -coreset.

□

Problem 7. 假设 $\alpha \in (0, 1]$. 假如我们将 (基本的) Morris 算法修改如下:

- (a) 初始化 $X \leftarrow 0$.
- (b) 对于每次更新, 以 $\frac{1}{(1+\alpha)^X}$ 的概率使 X 增加 1.
- (c) 对于查询, 输出 $\hat{n} = \frac{(1+\alpha)^X - 1}{\alpha}$.

记 X_n 为上述算法中 n 次更新以后的 X . 令 $\hat{n} = \frac{(1+\alpha)^{X_n} - 1}{\alpha}$.

- 计算 $\mathbb{E}[\hat{n}]$ 和 $\text{Var}[\hat{n}]$.
- 假设 $\epsilon, \delta \in (0, 1)$. 基于上述算法, 给出一个新算法, 使得新算法以至少 $1 - \delta$ 的概率输出一个估计值 $\hat{n}$, 满足 $|\hat{n} - n| \leq \epsilon n$. 说明你的算法的正确性与 (最坏) 空间复杂度 (即算法使用的比特数). 你的算法只需要满足以至少 $1 - \delta'$ 的概率, 其最坏空间复杂度为关于 $\frac{1}{\delta}, \frac{1}{\delta'}, \frac{1}{\epsilon}$ 和 $\log \log n$ 的多项式 (即 $\text{poly}(\frac{1}{\delta}, \frac{1}{\delta'}, \frac{1}{\epsilon}, \log \log n)$). (α 是常数)

提示：这道题的解答参考课上讲的 Morris 算法和 Morris+ 算法的证明

解. (1) 期望值计算

我们首先计算 $E[\tilde{n}]$ 。根据定义，有：

$$\Pr[X_{n+1} = x + 1 \mid X_n = x, x \in \mathbb{N}] = \frac{1}{(1 + \alpha)^x}$$

和

$$\Pr[X_{n+1} = x \mid X_n = x, x \in \mathbb{N}] = 1 - \frac{1}{(1 + \alpha)^x}$$

因此

$$\begin{aligned} E[(1 + \alpha)^{X_{n+1}}] &= \sum_{x \in \mathbb{N}} (1 + \alpha)^x \Pr[X_{n+1} = x] \\ &= \sum_{x \in \mathbb{N}} (1 + \alpha)^x \Pr[X_{n+1} = x \wedge (X_n = x \vee X_n = x - 1)] \\ &= \sum_{x \in \mathbb{N}} (1 + \alpha)^x (\Pr[X_{n+1} = x \wedge X_n = x] + \Pr[X_{n+1} = x \wedge X_n = x - 1]) \\ &= \sum_{x \in \mathbb{N}} ((1 + \alpha)^x \Pr[X_{n+1} = x \wedge X_n = x] + (1 + \alpha)^{x+1} \Pr[X_{n+1} = x + 1 \wedge X_n = x]) \\ &= \sum_{x \in \mathbb{N}} ((1 + \alpha)^x \Pr[X_{n+1} = x \mid X_n = x] + (1 + \alpha)^{x+1} \Pr[X_{n+1} = x + 1 \mid X_n = x]) \Pr[X_n = x] \\ &= \sum_{x \in \mathbb{N}} \left((1 + \alpha)^x \left(1 - \frac{1}{(1 + \alpha)^x} \right) + (1 + \alpha)^{x+1} \cdot \frac{1}{(1 + \alpha)^x} \right) \Pr[X_n = x] \\ &= \sum_{x \in \mathbb{N}} ((1 + \alpha)^x \Pr[X_n = x] + \alpha \Pr[X_n = x]) \\ &= \sum_{x \in \mathbb{N}} (1 + \alpha)^x \Pr[X_n = x] + \alpha \sum_{x \in \mathbb{N}} \Pr[X_n = x] \\ &= E[(1 + \alpha)^{X_n}] + \alpha. \end{aligned}$$

由于 $X_0 = 0$ ，即 $E[(1 + \alpha)^{X_0}] = 1$ ，通过归纳法易证 $E[(1 + \alpha)^{X_n}] = \alpha n + 1$ 。根据定义 $\tilde{n} = \frac{(1 + \alpha)^{X_n} - 1}{\alpha}$ ，故有：

$$E[\tilde{n}] = \frac{\alpha n + 1 - 1}{\alpha} = n$$

(2) 方差计算

我们采用相同方法计算 $E[(1 + \alpha)^{2X_n}]$:

$$\begin{aligned}
E[(1 + \alpha)^{2X_{n+1}}] &= \sum_{x \in \mathbb{N}} (1 + \alpha)^{2x} \Pr[X_{n+1} = x] \\
&= \sum_{x \in \mathbb{N}} (1 + \alpha)^{2x} \Pr[X_{n+1} = x \wedge (X_n = x \vee X_n = x - 1)] \\
&= \sum_{x \in \mathbb{N}} (1 + \alpha)^{2x} (\Pr[X_{n+1} = x \wedge X_n = x] + \Pr[X_{n+1} = x \wedge X_n = x - 1]) \\
&= \sum_{x \in \mathbb{N}} ((1 + \alpha)^{2x} \Pr[X_{n+1} = x \wedge X_n = x] + (1 + \alpha)^{2x+2} \Pr[X_{n+1} = x + 1 \wedge X_n = x]) \\
&= \sum_{x \in \mathbb{N}} ((1 + \alpha)^{2x} \Pr[X_{n+1} = x \mid X_n = x] + (1 + \alpha)^{2x+2} \Pr[X_{n+1} = x + 1 \mid X_n = x]) \Pr[X_n = x] \\
&= \sum_{x \in \mathbb{N}} \left((1 + \alpha)^{2x} \left(1 - \frac{1}{(1 + \alpha)^x} \right) + (1 + \alpha)^{2x+2} \cdot \frac{1}{(1 + \alpha)^x} \right) \Pr[X_n = x] \\
&= \sum_{x \in \mathbb{N}} ((1 + \alpha)^{2x} \Pr[X_n = x] + (2\alpha + \alpha^2)(1 + \alpha)^x \Pr[X_n = x]) \\
&= \sum_{x \in \mathbb{N}} (1 + \alpha)^{2x} \Pr[X_n = x] + (2\alpha + \alpha^2) \sum_{x \in \mathbb{N}} (1 + \alpha)^x \Pr[X_n = x] \\
&= E[(1 + \alpha)^{2X_n}] + (2\alpha + \alpha^2) E[(1 + \alpha)^{X_n}].
\end{aligned}$$

由于 $\tilde{n}^2 = \frac{(1+\alpha)^{2X_n} - 2(1+\alpha)^{X_n+1}}{\alpha^2}$, 可得:

$$\begin{aligned}
E[n \tilde{n} + 1^2] &= E \left[\frac{(1 + \alpha)^{2X_{n+1}} - 2(1 + \alpha)^{X_{n+1} + 1}}{\alpha^2} \right] \\
&= \frac{1}{\alpha^2} E[(1 + \alpha)^{2X_{n+1}}] - \frac{2}{\alpha^2} E[(1 + \alpha)^{X_{n+1}}] + \frac{1}{\alpha^2} \\
&= \frac{1}{\alpha^2} (E[(1 + \alpha)^{2X_n}] + (2\alpha + \alpha^2) E[(1 + \alpha)^{X_n}]) \\
&\quad - \frac{2}{\alpha^2} (E[(1 + \alpha)^{X_n}] + \alpha) + \frac{1}{\alpha^2} \\
&= \left(\frac{1}{\alpha^2} E[(1 + \alpha)^{2X_n}] - \frac{2}{\alpha^2} E[(1 + \alpha)^{X_n}] + \frac{1}{\alpha^2} \right) \\
&\quad + \frac{2 + \alpha}{\alpha} E[(1 + \alpha)^{X_n}] - \frac{2}{\alpha} \\
&= E[\tilde{n}] + \frac{2 + \alpha}{\alpha} (\alpha n + 1) - \frac{2}{\alpha} \\
&= E[\tilde{n}] + (\alpha + 2)n + 1.
\end{aligned}$$

我们可以通过数学归纳法证明 $E[\tilde{n}^2] = \frac{2+\alpha}{2}n^2 - \frac{\alpha}{2}n$ 。因此, 方差

$$\text{Var}[\tilde{n}] = E[\tilde{n}^2] - (E[\tilde{n}])^2 = \frac{\alpha}{2}(n^2 - n)$$

(3) 算法设计

- (a) 独立运行 $k \triangleq \lceil \frac{\alpha}{2\delta\epsilon^2} \rceil$ 个改进版 Morris 算法副本。收到更新请求时同步更新所有子程序。
- (b) 查询时调用每个子程序并记录输出 $\hat{n}_1, \hat{n}_2, \dots, \hat{n}_k$, 最终输出算术平均值 $\hat{n} = \frac{1}{k} \sum_{i=1}^k \hat{n}_i$ 。

(4) 正确性证明

由 (1) 可知, 经过 n 次更新后有 $E[\tilde{n}_1] = \dots = E[\tilde{n}_k] = n$ 且 $Var(\tilde{n}_1) = \dots = Var(\tilde{n}_k) < \frac{\alpha}{2}n^2$ 。由于 $\tilde{n} = \frac{1}{k} \sum_{i=1}^k \tilde{n}_i$ 且 $\tilde{n}_i$ 相互独立, 故:

$$E[\tilde{n}] = n, \quad Var[\tilde{n}] = \frac{1}{k^2} \sum_{i=1}^k Var[\tilde{n}_i] < \frac{\alpha}{2k}n^2$$

根据切比雪夫不等式, 当 $k \geq \frac{\alpha}{2\delta\varepsilon}$ 时:

$$Pr[|\tilde{n} - n| > \varepsilon n] \leq \frac{Var[\tilde{n}]}{\varepsilon^2 n^2} < \delta$$

这表明以至少 $1 - \delta$ 的概率满足 $|\tilde{n} - n| \leq \varepsilon n$ 。

(5) 空间复杂度说明

首先分析单个子程序的空间复杂度。由于每个计数器 X_i 最多递增 n 次, 且每次递增概率不超过 $\frac{\delta'}{nk}$, 因此:

$$Pr \left[X_i > \left\lceil \log_{1+\alpha} \frac{nk}{\delta'} \right\rceil \right] \leq \frac{\delta'}{k}$$

通过 union bound 可知, 所有 k 个子程序以至少 $1 - \delta'$ 的概率满足空间复杂度上界:

$$O \left(\log_{1+\alpha} \frac{nk}{\delta} \right) = O \left(\log_{1+\alpha} n + \log_{1+\alpha} k + \log_{1+\alpha} \frac{1}{\delta'} \right)$$

最终算法总空间复杂度为:

$$O \left(\frac{1}{\delta} \cdot \frac{1}{\varepsilon^2} \left(\log \log n + \log \log \frac{1}{\delta} + \log \log \frac{1}{\varepsilon} + \log \log \frac{1}{\delta'} \right) \right)$$

即空间复杂度为 $\text{poly}(\frac{1}{\delta}, \frac{1}{\delta'}, \frac{1}{\varepsilon}, \log \log n)$ 。

□

Problem 思考题.

1. 我们介绍了 rectified flow, 知道 rectified flow 试图求动态最优传输的概率路径, 从而使得扩散过程的推理步数降低。那么, 有没有其他的做法, 来让扩散方程的梯度项尽量和时间无关?
2. 大语言模型 (如 LLaMA、GPT) 通常需要在大规模语料上进行微调 (例如 instruction tuning 或 domain adaptation)。为了降低计算成本, 研究者尝试用 Coreset 方法从上千万条训练数据中选出一个具有代表性的子集 (如 5%) 进行训练。如果你无法访问模型梯度信息 (例如使用 GPT-4 API), 你如何设计一个“无监督”的 Coreset 策略? 你会如何定义“代表性”?